

Appendix: R Codes

R Code for Simulation I

```
#####
# Simulation I: Null Calibration
# Higher-Order Weighted Spacing Statistic
#####

rm(list = ls())

#####
# Simulation parameters
#####

N <- 5000
alpha <- 0.05

sample_sizes <- c(50, 100, 200, 500, 1000)
r_values <- c(1, 2, 3, 5, 8)

#####
# Weighting functions
#####

w_uniform <- function(x) {
  rep(1, length(x))
}

w_central <- function(x) {
  1 - (2 * x - 1)^2
}

w_tail <- function(x) {
  (2 * x - 1)^2
}

weight_functions <- list(
  Uniform = w_uniform,
  Central = w_central,
  Tail = w_tail
)

#####
# Higher-order weighted spacing statistic
#####

higher_order_statistic <- function(x, r, weight_function) {
```

```

n <- length(x)

# Ordered observations
x <- sort(x)

# Higher-order spacings:
#  $D_i^{(r)} = X_{(i+r)} - X_{(i)}$ 
#
# R indexing gives:
#  $x[(i+r)] - x[i]$ 
#
#  $i = 1, \dots, n-r$ 

D_r <- x[(r + 1):n] - x[1:(n - r)]

m <- length(D_r)

# Mean higher-order spacing
Dbar_r <- mean(D_r)

#####
# Numerator
#
#  $D_{(i+1)}^{(r)} - D_i^{(r)}$ 
#####

spacing_difference <- diff(D_r)

# Locations associated with the differences
# Use the midpoint of the corresponding observations
x_mid <- (x[1:(n - r - 1)] +
          x[(r + 2):n]) / 2

# Weight values
weights <- weight_function(x_mid)

numerator <- sum(
  weights * spacing_difference^2
)

#####
# Denominator
#####

denominator <- sum(
  (D_r - Dbar_r)^2
)

```

```

#####
# Return statistic
#####

statistic <- numerator / denominator

return(statistic)
}

#####
# Check the statistic for one sample
#####

set.seed(123)

x_test <- runif(100)

higher_order_statistic(
  x = x_test,
  r = 1,
  weight_function = w_uniform
)

#####
# Theoretical null constants
#####

theta_theoretical <- function(r, weight_name) {

  if (weight_name == "Uniform") {
    integral_w <- 1
  }

  if (weight_name == "Central") {
    integral_w <- 2 / 3
  }

  if (weight_name == "Tail") {
    integral_w <- 1 / 3
  }

  theta <- (2 / r) * integral_w

  return(theta)
}

#####

```

```

# Function to calculate null critical value
#
# The critical value is obtained from a separate Monte Carlo
# calibration sample.
#####

generate_null_critical_value <- function(
  n,
  r,
  weight_name,
  B_cal = 20000,
  alpha = 0.05
) {

  weight_function <- weight_functions[[weight_name]]

  statistics <- numeric(B_cal)

  for (b in seq_len(B_cal)) {

    x <- runif(n)

    statistics[b] <- higher_order_statistic(
      x = x,
      r = r,
      weight_function = weight_function
    )
  }

  # Upper-tail critical value
  critical_value <- quantile(
    statistics,
    probs = 1 - alpha,
    names = FALSE
  )

  return(critical_value)
}

#####
# Main Monte Carlo simulation
#####

set.seed(20260824)

results <- data.frame()

for (n in sample_sizes) {

```

```

cat("\n=====\\n")
cat("Sample size n =", n, "\\n")
cat("=====\\n")

for (r in r_values) {

  for (weight_name in names(weight_functions)) {

    cat(
      "Running: n =", n,
      ", r =", r,
      ", weight =", weight_name, "\\n"
    )

    weight_function <- weight_functions[[weight_name]]

    #####
    # Null critical value
    #####

    critical_value <- generate_null_critical_value(
      n = n,
      r = r,
      weight_name = weight_name,
      B_cal = 20000,
      alpha = alpha
    )

    #####
    # Monte Carlo replications
    #####

    test_statistics <- numeric(N)

    for (j in seq_len(N)) {

      # Generate null sample
      x <- runif(n)

      # Calculate statistic
      test_statistics[j] <-
        higher_order_statistic(
          x = x,
          r = r,
          weight_function = weight_function
        )
    }
  }
}

```

```

#####
# Rejection indicator
#####

rejection <- test_statistics > critical_value

empirical_size <- mean(rejection)

#####
# Monte Carlo standard error
#####

mcse <- sqrt(
  empirical_size *
  (1 - empirical_size) / N
)

#####
# Theoretical null constant
#####

theta <- theta_theoretical(
  r = r,
  weight_name = weight_name
)

#####
# Store results
#####

temp <- data.frame(
  n = n,
  r = r,
  Weight = weight_name,
  Theta = theta,
  Critical_Value = critical_value,
  Empirical_Size = empirical_size,
  MCSE = mcse
)

results <- rbind(results, temp)
}
}
}

#####
# Display results

```

```
#####

print(results)

#####
# Save results
#####

write.csv(
  results,
  "Simulation_I_Null_Calibration.csv",
  row.names = FALSE
)
```

R Code for Simulation II

```
#####
# Simulation II: Smooth Distributional Alternatives
# Higher-Order Weighted Spacing Statistic
#####

rm(list = ls())

#####
# Simulation parameters
#####

N <- 5000
alpha <- 0.05

sample_sizes <- c(50, 100, 200, 500, 1000)

r_values <- c(1, 2, 3, 5, 8)

#####
# Weight functions
#####

w_uniform <- function(x) {
  rep(1, length(x))
}

w_central <- function(x) {
  1 - (2 * x - 1)^2
}

w_tail <- function(x) {
  (2 * x - 1)^2
}
```

```

}

weight_functions <- list(
  Uniform = w_uniform,
  Central = w_central,
  Tail = w_tail
)

#####
# Higher-order spacing statistic
#####

higher_order_statistic <- function(
  x,
  r,
  weight_function
) {

  n <- length(x)

  # Ordered observations
  x <- sort(x)

  #####
  # Higher-order spacings
  #
  #  $D_i^{(r)} = X_{(i+r)} - X_{(i)}$ 
  #####

  D_r <- x[(r + 1):n] - x[1:(n - r)]

  m <- length(D_r)

  # Mean higher-order spacing
  Dbar_r <- mean(D_r)

  #####
  # Numerator
  #
  #  $D_{(i+1)}^{(r)} - D_i^{(r)}$ 
  #####

  spacing_difference <- diff(D_r)

  # Locations associated with the differences
  x_mid <- (
    x[1:(n - r - 1)] +
    x[(r + 2):n]
  )

```



```

) / 2

# Weight values
weights <- weight_function(x_mid)

numerator <- sum(
  weights * spacing_difference^2
)

#####
# Denominator
#####

denominator <- sum(
  (D_r - Dbar_r)^2
)

#####
# Return statistic
#####

statistic <- numerator / denominator

return(statistic)
}

#####
# Critical values from Simulation I
#
# These should be obtained from the null calibration.
#####

generate_null_critical_value <- function(
  n,
  r,
  weight_name,
  B_cal = 20000,
  alpha = 0.05
) {

  weight_function <- weight_functions[[weight_name]]

  statistics <- numeric(B_cal)

  for (b in seq_len(B_cal)) {

    # Generate observation from the null
    x <- runif(n)

```

```

# Calculate statistic
statistics[b] <- higher_order_statistic(
  x = x,
  r = r,
  weight_function = weight_function
)
}

# Upper-tail critical value
critical_value <- quantile(
  statistics,
  probs = 1 - alpha,
  names = FALSE
)

return(critical_value)
}

#####
# Smooth alternatives
#####

alternative_distributions <- list(

  Beta_2_2 = function(n) {
    rbeta(n, shape1 = 2, shape2 = 2)
  },

  Beta_3_3 = function(n) {
    rbeta(n, shape1 = 3, shape2 = 3)
  },

  Beta_2_5 = function(n) {
    rbeta(n, shape1 = 2, shape2 = 5)
  },

  Beta_5_2 = function(n) {
    rbeta(n, shape1 = 5, shape2 = 2)
  }

)

#####
# Monte Carlo simulation
#####

set.seed(20260824)

```

```

results_sim2 <- data.frame()

for (n in sample_sizes) {

  cat("\n===== \n")
  cat("Sample size n =", n, "\n")
  cat("===== \n")

  for (r in r_values) {

    for (weight_name in names(weight_functions)) {

      cat(
        "Calibrating: n =", n,
        ", r =", r,
        ", weight =", weight_name, "\n"
      )

      weight_function <- weight_functions[[weight_name]]

      #####
      # Null critical value
      #####

      critical_value <- generate_null_critical_value(
        n = n,
        r = r,
        weight_name = weight_name,
        B_cal = 20000,
        alpha = alpha
      )

      #####
      # Loop over smooth alternatives
      #####

      for (alternative_name in names(
        alternative_distributions
      )) {

        cat(
          "Running alternative:",
          alternative_name, "\n"
        )

        generate_sample <-
          alternative_distributions[[alternative_name]]

```

```
#####
# Monte Carlo test statistics
#####

test_statistics <- numeric(N)

for (j in seq_len(N)) {

  # Generate sample under alternative
  x <- generate_sample(n)

  # Calculate statistic
  test_statistics[j] <-
    higher_order_statistic(
      x = x,
      r = r,
      weight_function = weight_function
    )
}

#####
# Rejection indicator
#####

rejection <-
  test_statistics > critical_value

empirical_power <-
  mean(rejection)

#####
# Monte Carlo standard error
#####

mcse <- sqrt(
  empirical_power *
    (1 - empirical_power) / N
)

#####
# Store results
#####

temp <- data.frame(
  n = n,
  r = r,
  Weight = weight_name,
```

```

        Alternative = alternative_name,
        Critical_Value = critical_value,
        Empirical_Power = empirical_power,
        MCSE = mcse
    )

    results_sim2 <-
        rbind(results_sim2, temp)
    }
}
}
}

#####
# Display results
#####

print(results_sim2)

```

R Code for Simulation III

```

#####
# SIMULATION III
# Localized Clustering Alternatives
#
# Null density:
#      $f_0(x) = 1, \quad 0 < x < 1$ 
#
# Alternative:
#      $f_n(x) = f_0(x) [1 + c/\sqrt{n} * h(x)]$ 
#
#  $h(x) = q(x) - \int_0^1 q(u)du$ 
#
#  $q(x) = \exp(-(x-x_0)^2 / (2*\eta^2))$ 
#
# Therefore:
#      $\int_0^1 h(x) dx = 0$ 
#####

rm(list = ls())

#####
# 1. SIMULATION SETTINGS
#####

N <- 5000
alpha <- 0.05

```

```

sample_sizes <- c(50, 100, 200, 500, 1000)

r_values <- c(1, 2, 3, 5, 8)

x0_values <- c(0.25, 0.50, 0.75)

eta_values <- c(0.05, 0.10, 0.20)

c_values <- c(0.5, 1, 1.5, 2)

#####
# 2. WEIGHTING FUNCTIONS
#####

# Uniform weighting
weight_uniform <- function(x) {
  rep(1, length(x))
}

# Central weighting
weight_central <- function(x) {
  1 - (2 * x - 1)^2
}

# Tail weighting
weight_tail <- function(x) {
  (2 * x - 1)^2
}

#####
# 3. LOCALIZED PERTURBATION FUNCTION
#####

# Gaussian-shaped local perturbation
q_function <- function(x, x0, eta) {
  exp(
    -((x - x0)^2) / (2 * eta^2)
  )
}

#####
# 4. CALCULATE THE CENTERING CONSTANT

```

```

#
# Integral of q(x) over [0,1]
#####

get_centering_constant <- function(x0, eta) {

  integrate(
    function(u) {
      q_function(
        x = u,
        x0 = x0,
        eta = eta
      )
    },
    lower = 0,
    upper = 1
  )$value
}

#####
# 5. PRE-CALCULATE CENTERING CONSTANTS
#####

centering_constants <- list()

for (x0 in x0_values) {

  for (eta in eta_values) {

    key <- paste(
      x0,
      eta,
      sep = "_"
    )

    centering_constants[[key]] <-
      get_centering_constant(
        x0 = x0,
        eta = eta
      )
  }
}

#####
# 6. h_{x0,eta}(x)
#####

```

```

localized_h <- function(
  x,
  x0,
  eta,
  centering_constants
) {

  key <- paste(
    x0,
    eta,
    sep = "_"
  )

  q <- q_function(
    x = x,
    x0 = x0,
    eta = eta
  )

  h <- q - centering_constants[[key]]

  return(h)
}

#####
# 7. LOCALIZED ALTERNATIVE DENSITY
#####

localized_density <- function(
  x,
  n,
  c,
  x0,
  eta,
  centering_constants
) {

  h <- localized_h(
    x = x,
    x0 = x0,
    eta = eta,
    centering_constants = centering_constants
  )

  f <- 1 + (c / sqrt(n)) * h

```



```

    return(f)
}

#####
# 8. GENERATE SAMPLE FROM LOCALIZED ALTERNATIVE
#
# Rejection sampling on (0,1)
#####

generate_localized_sample <- function(
  n,
  c,
  x0,
  eta,
  centering_constants
) {

  #####
  # Construct a fine grid to obtain an upper bound
  #####

  grid <- seq(
    0,
    1,
    length.out = 2001
  )

  f_grid <- localized_density(
    x = grid,
    n = n,
    c = c,
    x0 = x0,
    eta = eta,
    centering_constants = centering_constants
  )

  f_max <- max(f_grid)

  #####
  # Rejection sampling
  #####

  sample_x <- numeric(0)

  while (length(sample_x) < n) {

    m <- max(

```

```

        n = length(sample_x),
        100
    )

    y <- runif(m)

    fy <- localized_density(
        x = y,
        n = n,
        c = c,
        x0 = x0,
        eta = eta,
        centering_constants = centering_constants
    )

    u <- runif(m)

    accepted <- y[
        u <= fy / f_max
    ]

    sample_x <- c(
        sample_x,
        accepted
    )
}

return(
    sample_x[1:n]
)
}

#####
# 9. HIGHER-ORDER WEIGHTED SPACING STATISTIC
#####

spacing_statistic <- function(
    x,
    r,
    weight_function
) {

    #####
    # Sample size
    #####

    n <- length(x)

```

```
#####
# Sort observations
#####

x_sorted <- sort(x)

#####
# r-th order spacings
#
#  $D_i^{(r)} = X_{(i+r)} - X_{(i)}$ 
#####

D_r <- x_sorted[
  (1 + r):n
] -
  x_sorted[
    1:(n - r)
  ]

#####
# Mean r-th order spacing
#####

D_bar <- mean(D_r)

#####
# Denominator
#####

denominator <- sum(
  (D_r - D_bar)^2
)

#####
# Difference of consecutive r-th order spacings
#####

differences <- diff(D_r)

#####
# Locations at which weights are evaluated
#####

weight_locations <- x_sorted[
  1:(n - r - 1)
]
```

```

weights <- weight_function(
  weight_locations
)

#####
# Numerator
#####

numerator <- sum(
  weights * differences^2
)

#####
# Final statistic
#####

statistic <- numerator / denominator

return(statistic)
}

#####
# 10. STORAGE OBJECT FOR CRITICAL VALUES
#####

critical_values <- list()

#####
# 11. CALCULATE NULL CRITICAL VALUES
#
# Critical values are calculated separately for:
#
#   n
#   r
#   weighting scheme
#
#####

cat("\n")
cat("=====\\n")
cat("CALCULATING NULL CRITICAL VALUES\\n")
cat("=====\\n")

for (n in sample_sizes) {

```

```

for (r in r_values) {

  if (r >= n) {
    next
  }

#####
# UNIFORM WEIGHTING
#####

null_uniform <- numeric(N)

for (b in 1:N) {

  x <- runif(n)

  null_uniform[b] <-
    spacing_statistic(
      x = x,
      r = r,
      weight_function = weight_uniform
    )
}

critical_values[
  paste(
    n,
    r,
    "Uniform",
    sep = "_"
  )
] <-
  quantile(
    null_uniform,
    probs = 1 - alpha,
    na.rm = TRUE
  )

#####
# CENTRAL WEIGHTING
#####

null_central <- numeric(N)

for (b in 1:N) {

```

```

x <- runif(n)

null_central[b] <-
  spacing_statistic(
    x = x,
    r = r,
    weight_function = weight_central
  )
}

critical_values[
  paste(
    n,
    r,
    "Central",
    sep = "_"
  )
] <-
  quantile(
    null_central,
    probs = 1 - alpha,
    na.rm = TRUE
  )

#####
# TAIL WEIGHTING
#####

null_tail <- numeric(N)

for (b in 1:N) {

  x <- runif(n)

  null_tail[b] <-
    spacing_statistic(
      x = x,
      r = r,
      weight_function = weight_tail
    )
}

critical_values[
  paste(
    n,
    r,
    "Tail",

```

```

        sep = "_"
    )
] <-
  quantile(
    null_tail,
    probs = 1 - alpha,
    na.rm = TRUE
  )

#####
# Progress message
#####

cat(
  "Critical values completed:",
  "n =", n,
  "r =", r,
  "\n"
)
}
}

```

```

#####
# 12. SIMULATION III RESULTS OBJECT
#####

```

```
results_sim3 <- data.frame()
```

```

#####
# 13. MAIN SIMULATION
#####

```

```

cat("\n")
cat("=====\\n")
cat("STARTING SIMULATION III\\n")
cat("=====\\n")

```

```

for (n in sample_sizes) {

  for (x0 in x0_values) {

    for (eta in eta_values) {

      for (c in c_values) {

```

```

for (r in r_values) {

  if (r >= n) {
    next
  }

#####
# THREE WEIGHTING SCHEMES
#####

for (
  weight_name in
  c(
    "Uniform",
    "Central",
    "Tail"
  )
) {

#####
# SELECT WEIGHT FUNCTION
#####

  if (
    weight_name == "Uniform"
  ) {

    weight_function <-
      weight_uniform

  } else if (
    weight_name == "Central"
  ) {

    weight_function <-
      weight_central

  } else {

    weight_function <-
      weight_tail
  }

#####

```



```

# GET CRITICAL VALUE
#####

critical_key <- paste(
  n,
  r,
  weight_name,
  sep = "_"
)

crit <- critical_values[
  critical_key
]

#####
# MONTE CARLO REPLICATIONS
#####

rejection <- logical(N)

for (b in 1:N) {

  #####
  # Generate alternative sample
  #####

  x <- generate_localized_sample(
    n = n,
    c = c,
    x0 = x0,
    eta = eta,
    centering_constants =
      centering_constants
  )

  #####
  # Calculate test statistic
  #####

  statistic <-
    spacing_statistic(
      x = x,
      r = r,
      weight_function =

```

```

        weight_function
    )

#####
# Test decision
#####

rejection[b] <-
    statistic > crit
}

#####
# EMPIRICAL POWER
#####

power <- mean(
    rejection
)

#####
# MONTE CARLO STANDARD ERROR
#####

mcse <- sqrt(
    power *
    (1 - power) /
    N
)

#####
# STORE RESULT
#####

results_sim3 <-
    rbind(
        results_sim3,
        data.frame(
            n = n,
            x0 = x0,
            eta = eta,
            c = c,
            r = r,
            Weight = weight_name,
            Power = power,

```

```

        MCSE = mcse
    )
)

#####
# Progress message
#####

cat(
  "Completed:",
  "n =", n,
  "| x0 =", x0,
  "| eta =", eta,
  "| c =", c,
  "| r =", r,
  "| Weight =", weight_name,
  "\n"
)
}
}
}
}
}

#####
# 14. DISPLAY RESULTS
#####

print(results_sim3)

#####
# 15. POWER + MCSE IN ONE COLUMN
#####

results_sim3$Power_MCSE <-
  sprintf(
    "%.4f (%.4f)",
    results_sim3$Power,
    results_sim3$MCSE
  )

#####
# 16. DISPLAY FORMATTED RESULTS

```

```
#####

print(results_sim3)

#####
# 17. SAVE COMPLETE RESULTS
#####

write.csv(
  results_sim3,
  "Simulation_III_Localized_Clustering.csv",
  row.names = FALSE
)

#####
# 18. SAVE R OBJECT
#####

save(
  results_sim3,
  file = "Simulation_III_Localized_Clustering.RData"
)

#####
# 19. FINAL MESSAGE
#####

cat("\n")
cat("=====\n")
cat("SIMULATION III COMPLETED\n")
cat("=====\n")
cat("Results saved as:\n")
cat("Simulation_III_Localized_Clustering.csv\n")
cat("Simulation_III_Localized_Clustering.RData\n")
cat("=====\n")
```

R Code for Simulation IV

```
#####
# OSCILLATORY ALTERNATIVE SAMPLE GENERATOR
#####

generate_oscillatory_sample <- function(
  n,
  c,
```

```

      k
    ) {

#####
# BASE DENSITY
#####

f0 <- function(x) {
  rep(1, length(x))
}

#####
# OSCILLATORY PERTURBATION
#####

h <- function(x) {
  sin(2 * pi * k * x)
}

#####
# PROPOSAL DENSITY
#####

density_function <- function(x) {

  f0(x) *
  (
    1 +
    c / sqrt(n) *
    h(x)
  )
}

#####
# REJECTION SAMPLING
#####

x <- numeric(n)

count <- 0

M <- 1 + abs(c) / sqrt(n)

while (count < n) {

```

```

y <- runif(1)

u <- runif(1)

accept_probability <-
  density_function(y) / M

if (u <= accept_probability) {

  count <- count + 1

  x[count] <- y
}
}

return(x)
}
#####
# SIMULATION IV: OSCILLATORY ALTERNATIVES
#####

#####
# SETTINGS
#####

sample_sizes <- c(
  50,
  100,
  200,
  500,
  1000
)

r_values <- c(
  1,
  2,
  3,
  5,
  8
)

k_values <- c(
  1,
  2,
  4,
  8

```

```

)

c_values <- c(
  0.5,
  1,
  1.5,
  2
)

N <- 5000

alpha <- 0.05

#####
# CRITICAL VALUES
#
# Critical values are obtained under U(0,1)
# separately for each n, r and weighting scheme.
#####

critical_values <- numeric()

#####
# CALCULATE NULL CRITICAL VALUES
#####

for (n in sample_sizes) {

  for (r in r_values) {

    if (r >= n) {
      next
    }

    #####
    # UNIFORM WEIGHTING
    #####

    null_uniform <- numeric(N)

    for (b in 1:N) {

      x <- runif(n)

      null_uniform[b] <-

```

```

    spacing_statistic(
      x = x,
      r = r,
      weight_function = weight_uniform
    )
  }

critical_key <- paste(
  n,
  r,
  "Uniform",
  sep = "_"
)

critical_values[critical_key] <-
  quantile(
    null_uniform,
    probs = 1 - alpha,
    na.rm = TRUE
  )

#####
# CENTRAL WEIGHTING
#####

null_central <- numeric(N)

for (b in 1:N) {

  x <- runif(n)

  null_central[b] <-
    spacing_statistic(
      x = x,
      r = r,
      weight_function = weight_central
    )
}

critical_key <- paste(
  n,
  r,
  "Central",
  sep = "_"
)

critical_values[critical_key] <-

```



```

quantile(
  null_central,
  probs = 1 - alpha,
  na.rm = TRUE
)

#####
# TAIL WEIGHTING
#####

null_tail <- numeric(N)

for (b in 1:N) {

  x <- runif(n)

  null_tail[b] <-
    spacing_statistic(
      x = x,
      r = r,
      weight_function = weight_tail
    )
}

critical_key <- paste(
  n,
  r,
  "Tail",
  sep = "_"
)

critical_values[critical_key] <-
  quantile(
    null_tail,
    probs = 1 - alpha,
    na.rm = TRUE
  )

#####
# PROGRESS MESSAGE
#####

cat(
  "Critical values completed:",
  "n =", n,
  "| r =", r,

```

```

        "\n"
    )
}
}

#####
# RESULTS OBJECT
#####

results_sim4 <- data.frame(
  n = numeric(0),
  k = numeric(0),
  c = numeric(0),
  r = numeric(0),
  Weight = character(0),
  Power = numeric(0),
  MCSE = numeric(0),
  stringsAsFactors = FALSE
)

#####
# SIMULATION IV
# OSCILLATORY ALTERNATIVES
#####

for (n in sample_sizes) {

  for (k in k_values) {

    for (c in c_values) {

      for (r in r_values) {

        if (r >= n) {
          next
        }

        #####
        # THREE WEIGHTING SCHEMES
        #####

        for (
          weight_name in
          c(
            "Uniform",

```

```

        "Central",
        "Tail"
    )
) {

#####
# SELECT WEIGHT FUNCTION
#####

if (
    weight_name == "Uniform"
) {

    weight_function <-
        weight_uniform

} else if (
    weight_name == "Central"
) {

    weight_function <-
        weight_central

} else {

    weight_function <-
        weight_tail

}

#####
# GET CRITICAL VALUE
#####

critical_key <- paste(
    n,
    r,
    weight_name,
    sep = "_"
)

crit <- critical_values[critical_key]

#####
# MONTE CARLO REPLICATIONS
#####

```

```

rejection <- logical(N)

for (b in 1:N) {

#####
# GENERATE OSCILLATORY ALTERNATIVE SAMPLE
#####

x <- generate_oscillatory_sample(
  n = n,
  c = c,
  k = k
)

#####
# CALCULATE TEST STATISTIC
#####

statistic <-
  spacing_statistic(
    x = x,
    r = r,
    weight_function = weight_function
  )

#####
# TEST DECISION
#####

rejection[b] <-
  statistic > crit
}

#####
# EMPIRICAL POWER
#####

power <- mean(
  rejection
)

```

```
#####
# MONTE CARLO STANDARD ERROR
#####

mcse <- sqrt(
  power *
  (1 - power) /
  N
)

#####
# STORE RESULT
#####

results_sim4 <-
  rbind(
    results_sim4,
    data.frame(
      n = n,
      k = k,
      c = c,
      r = r,
      Weight = weight_name,
      Power = power,
      MCSE = mcse,
      stringsAsFactors = FALSE
    )
  )

#####
# PROGRESS MESSAGE
#####

cat(
  "Completed:",
  "n =", n,
  "| k =", k,
  "| c =", c,
  "| r =", r,
  "| Weight =", weight_name,
  "\n"
)
}
}
}
}
```

```
}
```

```
#####  
# VIEW RESULTS  
#####
```

```
print(results_sim4)
```

```
#####  
# SAVE RESULTS  
#####
```

```
write.csv(  
  results_sim4,  
  "Simulation_IV_results.csv",  
  row.names = FALSE  
)
```